Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică
Program: Software Engineering

Operating Systems

Laboratory Work No. 11

The Race and the Semaphore

Track A: the real OS, from the outside (Linux)

Elaborated by:
st. gr. FAF-241
Ermicev Sofia

Verified by:
Patricia Reițman

Chișinău – 2026

Contents

1 Introduction **2**

2 Part 1. See the Race **2**
2.1 Program and execution 2
2.2 Observations 3

3 Part 2. Fix It: Mutex, Then Semaphore **3**
3.1 Mutex around the critical section 3
3.2 Semaphore used as a lock 3
3.3 Runtime comparison 4
3.4 Semaphore initialized to 2 4
3.5 Observations 5

4 Part 3. A Semaphore as a Signal **5**
4.1 Ordering the writer and reader 5
4.2 Observations 5

5 Part 4. A Semaphore as a Counter: Parking Lot **6**
5.1 Three places for six cars 6
5.2 One parking place 6
5.3 Observations 7

6 Conclusion **7**

1. Introduction

The purpose of this laboratory work is to observe a race condition in a shared counter and then prevent it using synchronization. The experiments examine a POSIX mutex, a semaphore used as a lock, a semaphore used as a signal, and a counting semaphore controlling access to a parking lot.

All programs were compiled and executed in Ubuntu under WSL2 as user `sofia`, in `/home/sofia/os-lab11`. The GCC compiler was version 13.3.0. Programs were compiled using `-Wall -pthread` without the `-O2` optimization flag. The terminal outputs below are the actual results obtained during the laboratory session and reported from the student's Ubuntu environment; they are text transcripts, not reconstructed screenshots.

2. Part 1. See the Race

2.1. Program and execution

In `race.c`, two pthreads each increment the same global `long counter` one million times. No synchronization protects the expression `counter++`. The expected final value is 2,000,000.

```
#define N 1000000
long counter = 0;
void *add(void *arg) {
    for (int i = 0; i < N; i++)
        counter++;
    return NULL;
}
```

Actual command output:

```
$ gcc -Wall -pthread -o race race.c
$ ./race
counter = 1149718 (expected 2000000)
$ ./race
counter = 1142062 (expected 2000000)
$ ./race
counter = 1231203 (expected 2000000)
```

The three runs lost 850,282, 857,938, and 768,797 increments respectively.

2.2. Observations

1. A non-atomic increment can be understood as three steps: load the old value, add one, then store the new value.
2. If both threads read the same old value before either stores its result, they can both write the same incremented value. Two attempted increments then increase the counter by only one.
3. The result changes between runs because thread interleavings vary with scheduling and CPU timing. The program has a data race and does not guarantee a particular final value.

3. Part 2. Fix It: Mutex, Then Semaphore

3.1. Mutex around the critical section

The program `mutex.c` uses `pthread_mutex_lock` and `pthread_mutex_unlock` around the increment, allowing only one thread at a time to modify the counter.

```
pthread_mutex_lock(&lock);
counter++;
pthread_mutex_unlock(&lock);
```

```
$ gcc -Wall -pthread -o mutex mutex.c
$ ./mutex
counter = 2000000 (expected 2000000)
$ ./mutex
counter = 2000000 (expected 2000000)
$ ./mutex
counter = 2000000 (expected 2000000)
```

3.2. Semaphore used as a lock

The program `sem.c` initializes a semaphore to 1. The `sem_wait` and `sem_post` calls surround the same critical section.

```
sem_init(&s, 0, 1);
// In each thread's loop:
sem_wait(&s);
counter++;
sem_post(&s);
```

```
$ gcc -Wall -pthread -o sem sem.c
$ ./sem
```

```
counter = 2000000 (expected 2000000)
$ ./sem
counter = 2000000 (expected 2000000)
$ ./sem
counter = 2000000 (expected 2000000)
```

3.3. Runtime comparison

The `time` command was used to compare all three implementations on the student's machine.

```
$ time ./race
counter = 1192634 (expected 2000000)
real    0m0.006s
user    0m0.003s
sys     0m0.001s

$ time ./mutex
counter = 2000000 (expected 2000000)
real    0m0.043s
user    0m0.041s
sys     0m0.021s

$ time ./sem
counter = 2000000 (expected 2000000)
real    0m0.073s
user    0m0.081s
sys     0m0.036s
```

The unsynchronized version was fastest in this measurement but incorrect. The synchronized versions were slower due to the overhead of acquiring and releasing synchronization primitives and waiting when necessary.

3.4. Semaphore initialized to 2

An experimental copy, `sem_two.c`, changed the initialization to `sem_init(&s, 0, 2)`, allowing two threads into the critical section at once.

```
$ ./sem_two
counter = 1953598 (expected 2000000)
$ ./sem_two
counter = 1982492 (expected 2000000)
$ ./sem_two
counter = 1984174 (expected 2000000)
```

3.5. Observations

1. Yes. Both `mutex` and `sem` produced exactly 2,000,000 on all three runs.
2. With the semaphore initialized to 2, both threads can enter the critical section concurrently, so the data race returns. The three observed results were below 2,000,000.
3. A locked version performs synchronization operations for every increment, which adds overhead and may cause contention or waiting. It trades speed for correctness.

4. Part 3. A Semaphore as a Signal

4.1. Ordering the writer and reader

In `order.c`, thread B starts before thread A but must not read the message until A has written it. The semaphore starts at zero; A signals completion with `sem_post`.

```
sem_init(&ready, 0, 0);
// Reader:
sem_wait(&ready);
// Writer, after strcpy(message, "hello"):
sem_post(&ready);
```

```
$ gcc -Wall -pthread -o order order.c
$ ./order
B: waiting for the message...
A: message written
B: I read "hello"
```

An experimental copy initialized the semaphore to 1 instead:

```
$ ./order_one
B: waiting for the message...
B: I read ""
A: message written
```

4.2. Observations

1. With initial value 1, B does not block at `sem_wait` and reads the empty global message buffer before A writes `hello`.

2. A semaphore initialized to zero represents an unavailable event or resource: a waiting thread cannot proceed until another thread signals it with `sem_post`.

5. Part 4. A Semaphore as a Counter: Parking Lot

5.1. Three places for six cars

In `parking.c`, six threads represent cars and a semaphore initialized to `PLACES = 3` counts free parking spaces. A mutex separately protects the `inside` counter and the corresponding print statement.

```
sem_wait(&places);
pthread_mutex_lock(&lock);
inside++;
printf("car %ld enters (inside: %d)\n", id, inside);
pthread_mutex_unlock(&lock);
// Later, after parking:
sem_post(&places);
```

Actual terminal output:

```
$ ./parking
car 1 enters (inside: 1)
car 2 enters (inside: 2)
car 3 enters (inside: 3)
car 1 leaves (inside: 2)
car 4 enters (inside: 3)
car 3 leaves (inside: 2)
car 5 enters (inside: 3)
car 2 leaves (inside: 2)
car 6 enters (inside: 3)
car 4 leaves (inside: 2)
car 5 leaves (inside: 1)
car 6 leaves (inside: 0)
```

The largest observed value of `inside` was 3.

5.2. One parking place

The experimental copy `parking_one.c` used `PLACES = 1`. Only one car entered at a time:

```
$ ./parking_one
car 1 enters (inside: 1)
```

```
car 1 leaves (inside: 0)
car 2 enters (inside: 1)
car 2 leaves (inside: 0)
car 3 enters (inside: 1)
car 3 leaves (inside: 0)
car 4 enters (inside: 1)
car 4 leaves (inside: 0)
car 5 enters (inside: 1)
car 5 leaves (inside: 0)
car 6 enters (inside: 1)
car 6 leaves (inside: 0)
```

5.3. Observations

1. The largest `inside` value was 3. The call `sem_wait(&places)` ensures that a car can enter only when a unit is available; `sem_post(&places)` gives it back upon leaving.
2. At `PLACES = 1`, the counting semaphore becomes a binary semaphore, allowing at most one car inside at a time and functioning as a lock for entry.

6. Conclusion

This laboratory demonstrated why unsynchronized access to shared data is unsafe: three runs of `race.c` gave three different incorrect results. A mutex and a semaphore initialized to 1 both preserved the expected counter value, at a measurable runtime cost. A semaphore initialized to 2 did not provide mutual exclusion. A semaphore initialized to 0 successfully enforced an order between writer and reader, while a semaphore initialized to 3 limited six parking threads to three simultaneous occupants. Together, these results show that the initial semaphore value determines whether it is used as a lock, a signal, or a counter.